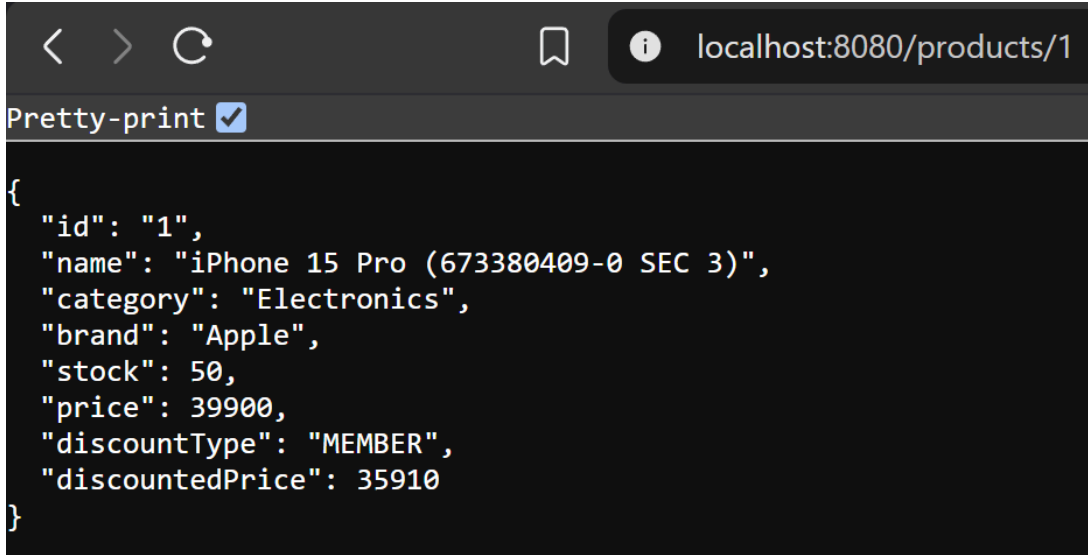


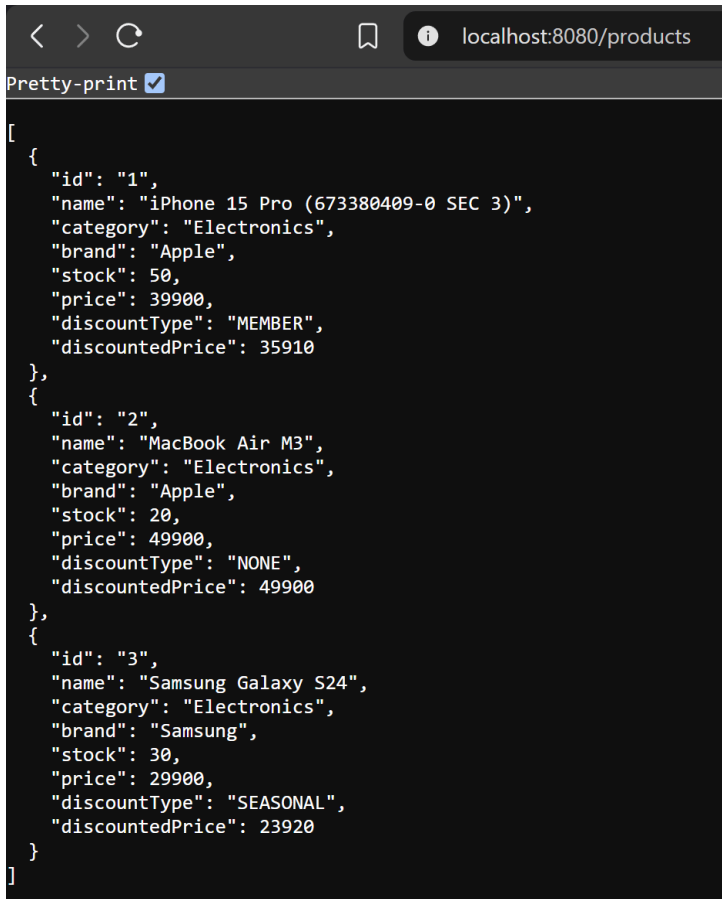
Lab10_Webclient

1. GET : <http://localhost:8080/products/1>



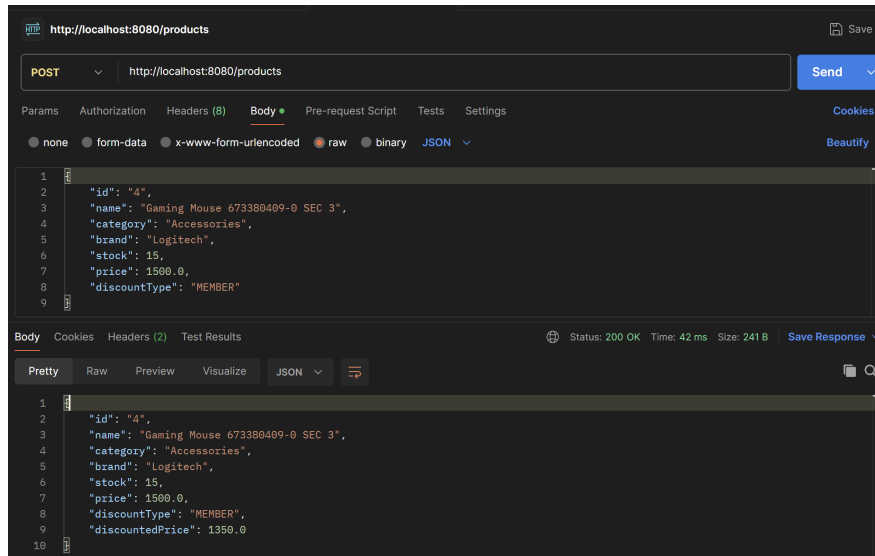
```
{
  "id": "1",
  "name": "iPhone 15 Pro (673380409-0 SEC 3)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900,
  "discountType": "MEMBER",
  "discountedPrice": 35910
}
```

2. GET : <http://localhost:8080/products>

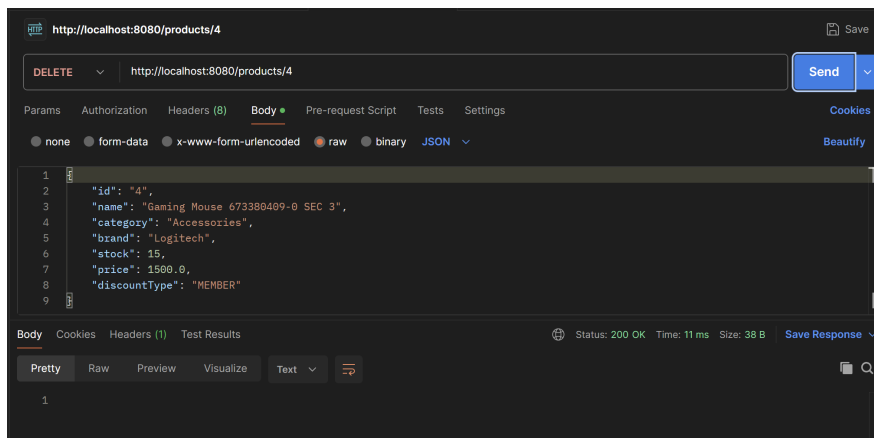


```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380409-0 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

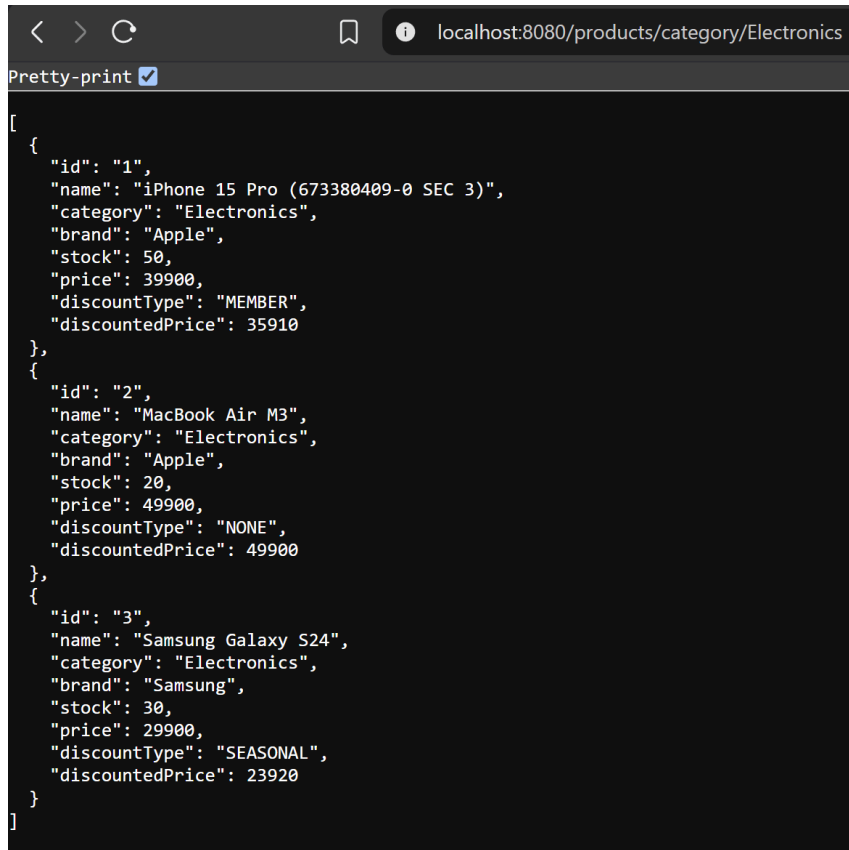
3. Post : <http://localhost:8080/products>



4. Delete : <http://localhost:8080/products/4>

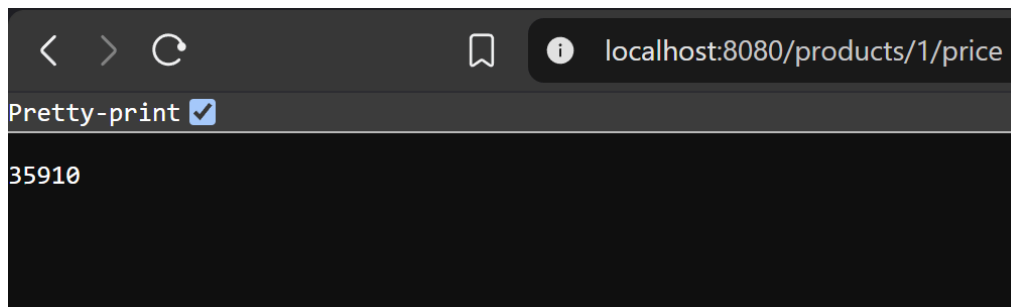


5. GET : <http://localhost:8080/products/category/Electronics>



```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380409-0 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

6. GET : <http://localhost:8080/products/1/price>



```
35910
```

คำถามท้ายLab

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ Blocking คือการทำงานที่โปรแกรมต้องรอให้งานหนึ่งเสร็จก่อน ถึงจะไปทำงานต่อได้ เช่น ถ้ากำลังรอข้อมูลจาก Server ก็ต้องรอจนกว่าจะได้ข้อมูลกลับมา ส่วน Reactive Programming จะไม่จำเป็นต้องหยุดรอ แต่สามารถไปทำงานอื่นก่อนได้ เมื่อข้อมูลพร้อมแล้วจึงกลับมาทำงานต่อ ทำให้เหมาะกับระบบที่ต้องรับ Request หลาย ๆ Request พร้อมกัน และช่วยให้ทรัพยากรได้มีประสิทธิภาพมากขึ้น

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono และ Flux ใช้สำหรับรับส่งข้อมูลแบบ Reactive โดย Mono ใช้เมื่อข้อมูลที่ให้มี 0 หรือ 1 ค่า เช่น การค้นหา Product ตาม ID เพราะจะได้สินค้ากลับมาแค่หนึ่งรายการ ส่วน Flux ใช้เมื่อข้อมูลมีได้หลายค่า เช่น การดึง Product ทั้งหมด หรือค้นหาสินค้าตาม Category เพราะผลลัพธ์สามารถมีสินค้าได้หลายรายการ

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ WebClient ใช้สำหรับส่ง Request ไปยัง Server หรือ API โดย .get() ใช้ระบุว่าส่งแบบ GET, .uri() ใช้กำหนด URL ที่ต้องการเรียก, .retrieve() ใ้รับ Response ที่ Server ส่งกลับมา และ .bodyToMono() ใช้เมื่อผลลัพธ์มีหนึ่งค่า ส่วน .bodyToFlux() ใช้เมื่อผลลัพธ์มีหลายค่า เช่น การดึง Product หนึ่งรายการจะใช้ bodyToMono(Product.class) แต่ถ้าดึง Product ทั้งหมดจะใช้ bodyToFlux(Product.class)

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ตอบ

Repository

```
Java
package com.example.lab10.repository;

import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ProductRepository {
    private final Map<String, Product> store = new ConcurrentHashMap<>();
    public ProductRepository() {

        store.put("1", new Product(
            "1",
            "iPhone 15 Pro (673380409-0 SEC 3)",
            "Electronics",
            "Apple",
            50,
        ));
    }
}
```

```

        39900.0,
        "MEMBER"
    ));

    store.put("2", new Product(
        "2",
        "MacBook Air M3",
        "Electronics",
        "Apple",
        20,
        49900.0,
        "NONE"
    ));

    store.put("3", new Product(
        "3",
        "Samsung Galaxy S24",
        "Electronics",
        "Samsung",
        30,
        29900.0,
        "SEASONAL"
    ));
}

public Mono<Product> findById(String id) {

    Product product = store.get(id);

    if (product == null) {
        return Mono.empty();
    }

    return Mono.just(product);
}

public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

public Mono<Product> save(Product product) {

    store.put(product.getId(), product);

    return Mono.just(product);
}

```

```

    }

    public Mono<Void> deleteById(String id) {

        store.remove(id);

        return Mono.empty();
    }

    public Flux<Product> findByCategory(String category) {

        return findAll()
            .filter(product ->
                product.getCategory().equalsIgnoreCase(category)
            );
    }
}

```

อธิบาย

ProductRepository เป็นส่วนที่ทำหน้าที่จัดการข้อมูลสินค้า โดยใน Lab นี้ยังไม่ได้เชื่อมต่อกับฐานข้อมูลจริง แต่เก็บข้อมูลไว้ในหน่วยความจำด้วย **ConcurrentHashMap** และใช้ **Mono** กับ **Flux** เพื่อรองรับการทำงานแบบ Reactive Programming ภายใน Repository มีเมธอดสำหรับจัดการข้อมูลสินค้า ได้แก่ **findById()** สำหรับค้นหาสินค้าจาก ID, **findAll()** สำหรับแสดงสินค้าทั้งหมด, **save()** สำหรับเพิ่มหรือแก้ไขสินค้า, **deleteById()** สำหรับลบสินค้า และ **findByCategory()** สำหรับค้นหาสินค้าตามหมวดหมู่ โดย **Mono** ใช้กับข้อมูลที่มีได้ 0-1 รายการ ส่วน **Flux** ใช้กับข้อมูลที่สามารถมีได้หลายรายการ

Service

```

Java
package com.example.lab10.service;

import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.UUID;

@Service
public class ProductService {
    private final ProductRepository repository;
}

```

```

public ProductService(ProductRepository repository) {
    this.repository = repository;
}

public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(
            Mono.error(
                new RuntimeException("Product not found: " + id)
            )
        );
}

public Flux<Product> getAll() {
    return repository.findAll();
}

public Mono<Product> save(Product product) {

    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }

    return repository.save(product);
}

public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(product -> product.getDiscountedPrice());
}
}

```

อธิบาย

ProductService เป็นส่วนที่ทำหน้าที่จัดการ Business Logic ของระบบ โดยรับคำสั่งจาก Controller แล้วเรียกใช้งาน

ProductRepository เพื่อจัดการข้อมูลสินค้า ภายในมีเมธอดสำหรับค้นหาสินค้าตาม ID, ดึงสินค้าทั้งหมด, เพิ่มหรือบันทึกสินค้า, ลบสินค้า และค้น

หาตามหมวดหมู่ นอกจากนี้ หากเพิ่มสินค้าโดยไม่มี ID ระบบจะสร้าง ID อัตโนมัติด้วย **UUID** และมีเมธอดสำหรับคำนวณราคาสินค้าหลังหักส่วนลด โดยผลลัพธ์จะถูกส่งกลับในรูปแบบ **Mono** หรือ **Flux** ตามแนวทาง Reactive Programming

Controller

Java

```
package com.example.lab10.controller;

import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }

    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }

    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) {
```



```

        return service.getByCategory(category);
    }

    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}

```

อธิบาย

ProductController เป็นส่วนที่ทำหน้าที่รับ Request จากผู้ใช้งานผ่าน REST API แล้วส่งต่อไปยัง ProductService เพื่อประมวลผล โดยกำหนด URL หลักเป็น /products ภายในมี API สำหรับค้นหาสินค้าตาม ID, แสดงสินค้าทั้งหมด, เพิ่มสินค้าใหม่, ลบสินค้า, ค้นหาสินค้าตามหมวดหมู่ และดูราคาล้างหักส่วนลด ข้อมูลที่ได้จะถูกส่งกลับในรูปแบบ Mono หรือ Flux ทำให้ Controller รองรับการทำงานแบบ Reactive Programming เช่นเดียวกับส่วน Service และ Repository